Programming Languages and Tools

1. Language summary

Language or format	Where used	Why it is used	Advantages in this system
Arduino C++	`Games`, `libraries/PixelGridcore`, `tests`, vendored libraries	Firmware, game logic, hardware abstraction, host tests.	Direct access to Arduino APIs, efficient embedded execution, reusable classes/structs, host compilation for selected logic.
Markdown	`README.md`, `Tutorial`, `tests`, `docs/evidence-pack`	Human-readable documentation.	Portable, versionable, renders well on GitHub, supports Mermaid diagrams.
YAML	`.github/workflows/tetris-tests.yml`	CI workflow configuration.	Declarative automation for build and test checks.
Arduino library metadata	`libraries/PixelGridcore/library.properties` and vendored library properties	Library discovery by Arduino tooling.	Enables Arduino IDE sketchbook workflow.

2. Frameworks and APIs

2.1 Arduino framework

- `setup()` and `loop()` entry points.
- GPIO input/output via `pinMode`, `digitalRead`, and pin constants.
- Timing via `millis()` and `delay()`.
- Serial communication via `Serial`.
- Random seeding and random piece generation.

Advantages:

- Simple embedded programming model.

- Broad board and library ecosystem.
- Appropriate for educational and rapid prototyping contexts.

2.2 Adafruit NeoPixel

Used for LED strip/matrix output through `Adafruit_NeoPixel`. It appears in game sketches, renderers, PixelGridCore, and tests stubs.

Advantages:

- Direct support for addressable RGB LEDs.
- Provides colour packing helpers.
- Well-known Arduino ecosystem library.

2.3 PixelGridCore local library

Used by Tetris, Breakout, and hardware sketches to abstract display hardware.

Advantages:

- Encapsulates grid coordinate mapping.
- Reuses LCD digit/panel logic across games.
- Reduces direct LED-index manipulation in game code.

2.4 ESP32 Wi-Fi, HTTP, TLS, and EAP APIs

Used in `Games/Tetris/NetSubmit.h`:

- `WiFi.h` for station mode and connection status.
- `WiFiClientSecure.h` for HTTPS client transport.
- `HTTPClient.h` for JSON POST requests.
- `esp_wifi.h` and `esp_eap_client.h` for enterprise Wi-Fi/EAP configuration.

Advantages:

- Enables network-connected score submission.
- Supports campus-style enterprise Wi-Fi environments.
- Allows the device to integrate with external services.

2.5 mbedTLS

Used in `Games/Tetris/NetSubmit.h` for:

- HMAC-SHA256 message authentication.
- Base64 encoding before conversion to base64url.

Advantages:

- Provides cryptographic primitives commonly available in ESP32 toolchains.
- Supports signed score submission to reduce tampering.

2.6 GitHub Actions

Used in `.github/workflows/tetris-tests.yml` to build and run host-side Tetris tests on push and pull request events affecting relevant paths.

Advantages:

- Automated regression checks.
- Source-control-aware CI triggers.
- Repeatable build command for host tests.

3. Libraries and packages

Library/package	Location	Use
PixelGridCore	`libraries/PixelGridcore`	Local shared grid, shape, button, LCD digit, and LCD panel abstractions.
Adafruit NeoPixel	`libraries/Adafruit_NeoPixel`	Addressable LED control.
FastLED	`libraries/FastLED`	Vendored LED library; not all usages are visible in project game modules, but available in sketchbook libraries.
Firmata	`libraries/Firmata`	Vendored Arduino communication library; available in sketchbook libraries.
Arduino/ESP32 core libraries	Included by sketches and network module	GPIO, serial, Wi-Fi, HTTP, TLS, timing, FreeRTOS task creation, random values.
mbedTLS	Included by `NetSubmit.h`	HMAC and base64 support.

4. Build systems

4.1 Arduino IDE sketchbook workflow

Typical build/upload path:

1. Open a sketch such as `Games/Tetris/Tetris.ino` or `Games/Breakout/Breakout.ino` in Arduino IDE.
2. Select the appropriate board and port.
3. Compile and upload.

4.2 Host C++ test build

The host test build uses:

```
g++ -std=c++17 -I tests/stubs -I Games/Tetris tests/tetris_game_tests.cpp -o
tests/tetris_game_tests
./tests/tetris_game_tests
```

This compiles selected Tetris logic without the Arduino hardware by using stubs from `tests/stubs`.

4.3 Continuous integration

`.github/workflows/tetris-tests.yml` runs the same host build and test commands on `ubuntu-latest` when relevant files change.

5. Testing tools

Tool	Location	Purpose
`g++`	Local and CI test command	Compile C++17 host tests.
Custom C++ assertions	`tests/tetris_game_tests.cpp`	Validate Tetris behaviours without a full testing framework.
Arduino/renderer stubs	`tests/stubs`	Replace hardware dependencies during host compilation.
GitHub Actions	`.github/workflows/tetris-tests.yml`	Automate build and test execution.

6. Deployment tooling

Deployment tooling is Arduino IDE-oriented:

- Arduino sketch upload is documented in `README.md` and `Tutorial/setup_upload.md`.
- Local libraries are vendored for sketchbook discovery.
- No Dockerfile, Kubernetes manifests, PlatformIO configuration, or Arduino CLI configuration is present.
- No backend deployment configuration is present.

7. IDE and toolchain assumptions

- Arduino IDE is the primary firmware development and upload tool.
- A compatible Arduino/ESP32 board package is installed manually.
- A USB data cable is used to upload sketches.
- `g++` with C++17 support is used for host tests.
- Git and GitHub are used for source control and CI.
- Visual Studio workspace metadata under `.vs` suggests at least one contributor used Visual Studio locally.

8. Source control awareness

- A `.git` directory in the working copy.
- `.gitignore` for ignored files.
- A GitHub Actions workflow under `.github/workflows`.
- Path-filtered CI triggers for Tetris and tests.
- Existing documentation and tests that reference CI behaviour.

Recommended source-control improvements:

- Add a dependency/version policy for vendored libraries.
- Consider excluding local IDE workspace state if it is not intentionally shared.

- Add example configuration files for required secrets without committing actual secrets.
- Add branch protection requiring the Tetris test workflow before merge.